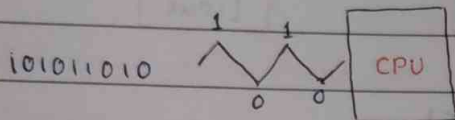


Web Technologies

- * Desktop Application :- Needs to be installed on each machine.
- * Web Application :- Is hosted on server and everyone can access using Browser.

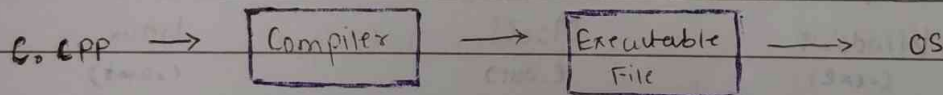
Machine - Level Instructions



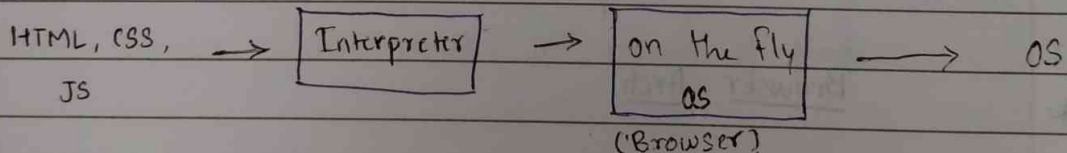
* Types of languages :-

- 1) Compiled → Compilation takes place to convert (Source → machine readable code)
- 2) Interpreted → Code is executed line-by-line on the fly.

(Platform dependent)



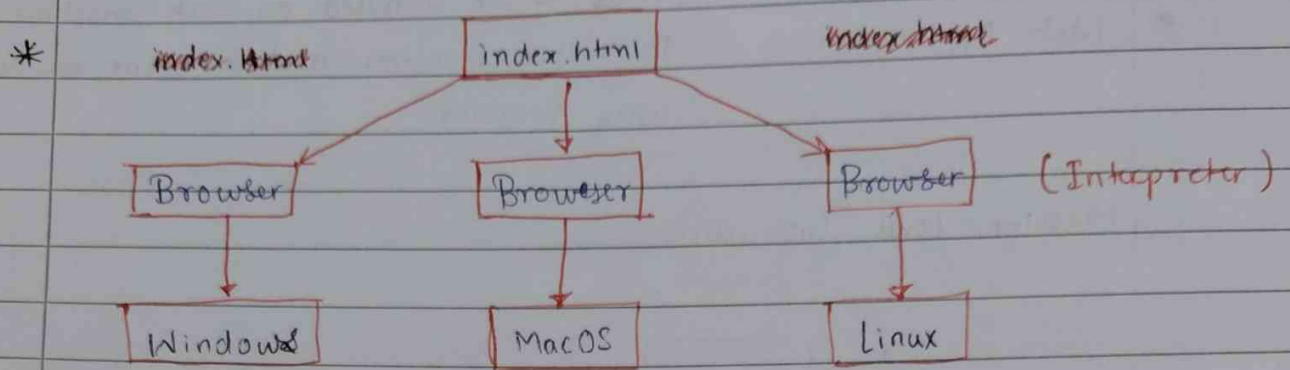
(Platform independent)



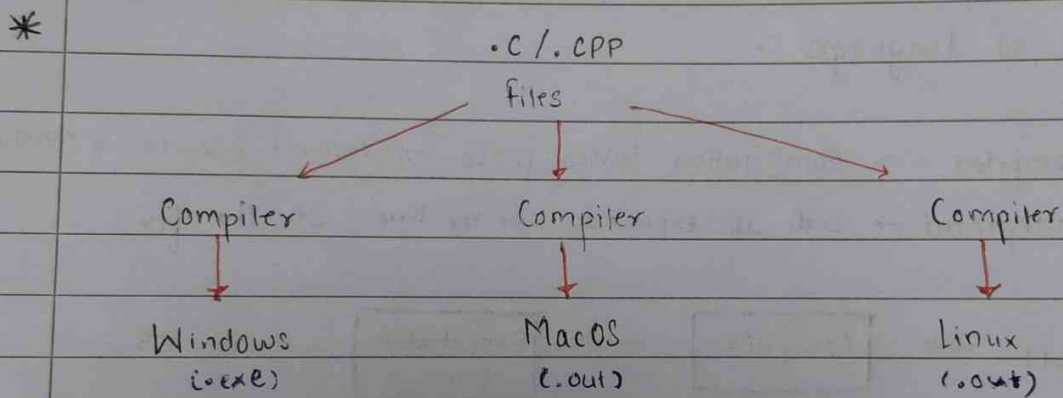
- * ~~Interpreted~~ Interpreted languages are executed inside a browser which provides a run-time environment.

- * Browser is platform ~~not~~ dependent and it ~~provides~~ provides a environment to run the interpreted code on the machine.

* Browser Architecture

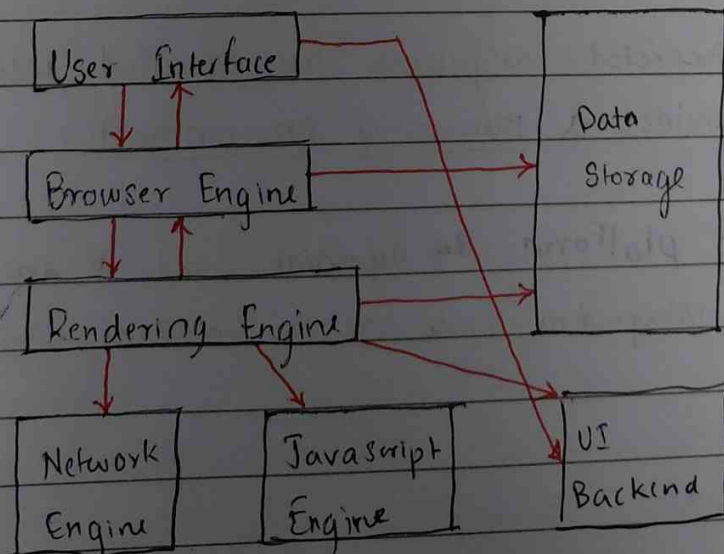


Interpreted Flow



Compiled Flow

* Browser Arch.



* Rendering Engine

Safari --> Webkit

Chrome --> Blink

Firefox --> Gecko

* JavaScript Engine

Safari --> JavaScript Core

Chrome --> V8

Firefox --> SpiderMonkey

* Web Architecture

* Framework :- i) Set of Libraries

ii) Provides a structure that provides us methods to develop application easily.

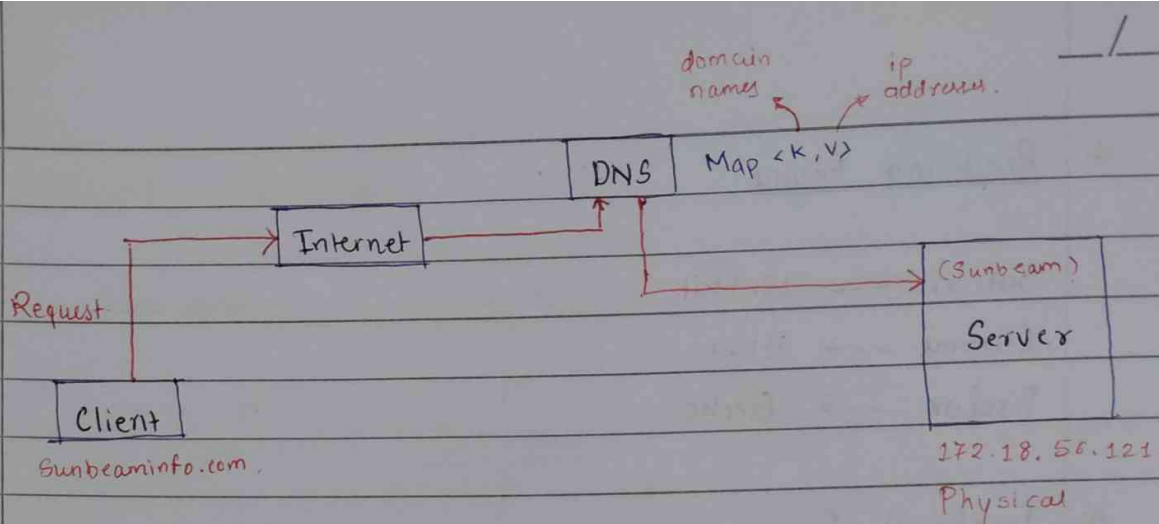
* So, just to work with JavaScript code we'll extract out the JS Engine from Browser, which results in creation of Node.

* Server - A powerful physical machine, heavily configured with hardware, connected to internet (having an IP address)

- used to deploy web applications (amazon.com, flipkart.com, etc)

* Client - uses a domain name to access the web application

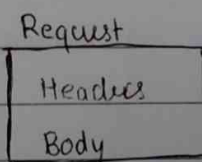
- the domain names are then mapped to the server's IP using the DNS



* Any server that is self-hosted on our machine, it's known as localhost.

* **Request:-**

it is an object passed by client, consisting of Headers and Body.



* **URL** :- Uniform Resource Locator

* HTTP uses port no 80

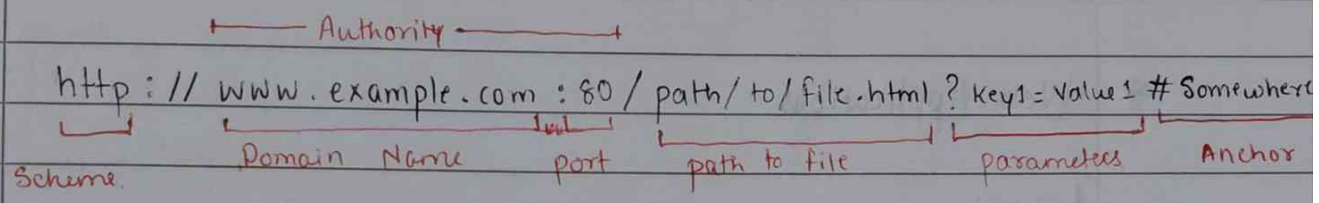
* HTTPS uses port no 443

(Secure → data is encrypted & send to server)

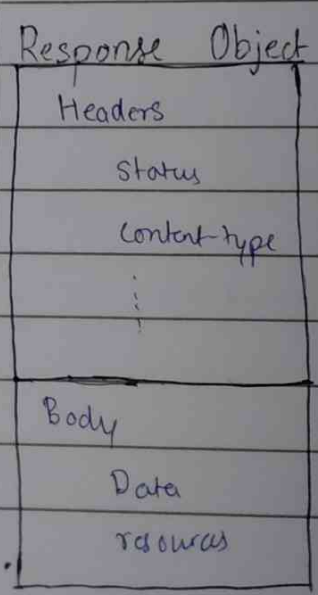
* To make the server as a server, we need to install an application called WebServer.
And,

Depending on requirements, we can install multiple server applications.

- * While configuring web applications, using HTTPS, we need to provide a Security Certificate, at the time deploying.
- * MySQL is a database Server, which defaults run on port 3306, using the HTTP protocol.
- * Live Server is a dummy server, using HTTP configured on port 5500.
- * Port Numbers will help us navigate across the Server applications present on the Web Server



- * **Anchor** :- used to navigate to a part of a particular page.
- * **Authority** :- Combination of domain name and the port number and is the first thing present in Headers Object.
- * Request Headers - passed on from internet → DNS → Server
- * In response, the Server returns a **Response**.



Methods

- GET → To fetch the data from Server.
- POST → Adding the data to the server.
- PUT → Update the resource on the Server.
- DELETE → To delete the resource from the Server.

Status Codes Series

- i) ~~100~~ 1XX → Informational codes
- ii) 2XX → Success message codes (Request processed)
- iii) 3XX → Further Actions Required from client (Redirection)
- iv) 4XX → Client Errors
- v) 5XX → Server Errors

Web Applications

HTML

- * To mention the type/version of HTML we use a `<!DOCTYPE html>` tag, but now it just supports the latest HTML version.

Tag:- any word included inside angled brackets (<>), is called a tag or also known as Element.

Types of Tags:-

- 1) Root Tag
- 2) Opening Tag
- 3) Closing Tag
- 4) Empty tag

* Root tag :-

Any particular tag which includes multiple different tags within itself, it is called a Root tag.

For ex:- HTML has only one Root tag, i.e. a `<html></html>` tag.

Every opening tag must have a closing tag

* Opening and Closing tag :-

Opening tag acts as the entry point to include the body and the closing tag acts as a closing point.

* Empty tag :-

It is a tag which doesn't have a closing tag and no body.

For ex:- `<img />`

Structure :-

```
<html>
  <head>
    <!-- Metadata -->
  </head>
  <body>
    <!-- contents -->
  </body>
</html>
```

* Tags used inside <head> tag

- | | |
|----------|-----------|
| 1) meta | 4) Script |
| 2) title | 5) link |
| 3) style | 6) base |

* Types of tags used in <body> tag

- | | |
|-------------|-----------|
| 1) Textual | 4) Table |
| 2) Resource | 5) form |
| 3) List | 6) Anchor |

* title tag

<title> My Webpage </title>

→ used to show the title for our webpage.

* link tag

<link rel=" " href=" " >

can be styled⁴
or icon, etc.

↳ path/link to the
resource.

→ used to link resource in the webpage

* Every tag has some attributes, there are 3 attributes common among every tag, those are

- id
- class
- style

//_

* **Attribute**:- Information kept inside the tag in the key-value pair.

* **meta tag**:- used to provide the metadata for the webpage and also we can write our own meta tags.

`<meta name="viewport" content="width=device-width initial-scale=1.0" />`

<u>name="viewport"</u>	<u>content="width=device-width"</u>	<u>initial-scale=1.0"</u>
Selects/displays the display which we are currently seeing	Sets the or adjust the content based on the device-width	initial zoom scale of the display.

* **Header tags** (type of textual tag)

→ h1, h2, h3, ..., h6

* **Div tags** are used to ~~logically~~ group logically equivalent elements

* **P tags** should be used to display textual (long) data.

* **List tag**

- i) Ordered List
- ii) Unordered List

i) `<ol type="a">`
 `<li> DAC </li>`
 `<li> DMC </li>`
`</ol>`

ii) `<ul>`
 `<li> DAC </li>`
 `<li> DMC </li>`
`</ul>`

* **Definition list**

```
<dl>
  <dt> HTML </dt>
  <dd> Hyper Text Markup Language </dd>
</dl>
```

* Formatting tag

`<b> Hello </b>` --> bold

`<u> Hello </u>` --> underline

`<i> Hello </i>` --> italic

→ All the formatting tags are inline tags

* Strike tag

`<p> offer price Rs 999 <strike> Rs 1299 </strike> </p>`

↓

~~Rs 1299~~

* Super tag

`<p> 16 <sup>th </sup> Aug 2025 </p>`

(16th Aug 2025)

* Subscript tag

`<p> Log <sub>10 </sub> 10 </p>`

(Log₁₀ 10)

Deprecated

* Font tag

`<p> <font> Sunbeam </font> </p>`

* Table (<table> </table>)

<thead> -> heading of column of table

<tbody> -> content of columns

<tfoot> -> Footer of the table

PID	Name	Price	thead
1	Pen	10	
2	Pencil	5	tbody
3	Book	50	
	Total	65	tfoot

--- table

<thead

<tr>

<th> Pid </th>

<th> Name </th>

<th> Price </th>

</tr>

</thead>

<tbody>

<tr>

<td> 1 </td> <td> Pen </td> <td> 10 </td>

</tr>

<tr> <td> 2 </td> <td> Pencil </td> <td> 5 </td>

</tr>

<tr> <td> 3 </td> <td> Book </td> <td> 50 </td>

</tr>

</tbody>

---- /table.

* for the data of column we use the tag (<td> </td>),
for ex:-

<tr>

<td> 1 </td>

</tr>

* to give a border to the table we can give a attribute called border

<table border = 1>

* to give the size of 2 columns, we use a attribute called colspan

```
<tr>  
  <td colspan="2"> Total </td> <td> 97 </td>  
</tr>
```

* tr represents table row

* thead represents table header, used for column headings

* th represents heading of a table column

* td represents data of the table column

* rowspan attribute

To show the repeated row only once, we can use rowspan.

```
<tr>  
  ...  
  <td rowspan="5"> Stationary </td>  
  ...  
</tr>
```

* Resource Tags

* image tag

```
  
path to file ↙ ↘ description / name
```

* another attribute → title = " "

* audio tag

```
<audio src=" " > Audio </audio>
```

to play/pause/control the audio, we compulsorily use the controls, (controls) attribute.

* video tag

```
<video src=" " controls width="200px"> </video>
```

controls attribute same as above.

* anchor tag

to go to a particular position or on the specific heading/word or navigate across the webpage, we use anchor tag.

* form tag

* to create a form and send back the data to server inputted by data.

```
<form>
```

```
<label for=" " > Name </label>
```

```
<input type="text" />
```

```
<btn> Submit </btn>
```

```
</form>
```

*/ When method is GET, data needs to be passed through query parameters

*/ when method is POST, data needs to be passed in the body of Request Object.

*/ HTML pages are by default designed for GET requests, and if we use a POST request, it'll give 405 method not Allowed error.

```
<input type='text' name='age' />
```

↓

acts as a key for the parameter, if we don't give this, it'll not show the query params in URL.

*/ type='tel' → Phone

type='email' → email

type='password' → Password

*/ required attribute, to make a field compulsory to fill/enter

*/ Validations on length
i) maxlength='10'
ii) minlength='6'

*/ Radio Button:-
<input type='radio' name='gender'> Male
<input type='radio' name='gender'> Female

↓

here the Key (name) should be same, to select just one of the options.

and to identify which option is selected, we use the 'Value' attribute.

```
<input type='radio' name='gender' value='male'> Male  
<input type='radio' name='gender' value='female'> Female
```

//_

* for checkbox :-

```
<input type="checkbox" name="checkbox" value="CPP"/> CPP
```

* Dropdown

```
<Select name='city'>
  <option> Pune </option>
  <option> Mumbai </option>
  <option> Bangalore </option>
</Select>
```

* For date

* for time

```
<input type='date' />      <input type='time' />
```

* For datetime

```
<input type='datetime-local' />
```

* For Saving form

```
<input type='submit' value='save' />
```

* For Clearing form

```
<input type='reset' value='clear' />
```

* For multiple line input

```
<textarea name = " " id = " " #> </textarea>
```

* Common Tags

Header, footer, section, article, nav, aside

- These tags are used to segregate the entities in logically equivalent groups.

* Header tag

- used to keep the headings, (divs, images, paras) of a webpage.

* Nav tag

- used to navigate to different pages of the same website, (internally can be using <a> tags)

* Section tag

- used to separate different, different data related to the same webpage.

* Footer tag

- used to represent/display the summary of the webpage

* Article tag

- used to indirectly relate to a webpage or
- to add some extra info on a webpage.

i.e. the data kept inside the tag is indirectly related to the webpage.

* aside tag

- used to keep data which is not directly or indirectly related to the webpage
- for examples :- ads on b.s. of a ~~website~~ website)

CSS (Cascading Stylesheets)

* Inline CSS

```
<p style="color: red;" > ----- </p>
```

- cant write in each p tag for color, so we use internal CSS.

* Internal CSS

Declaration block ({})

Selector `<style>`

`{`

`p {`

`color: red;` Value

`}` CSS Property

`</style>`

- will apply to all the paragraphs in the html page.

- written in the same HTML file

CSS Rule :- Selector + Declaration

(p) ({ color: red; })

* External CSS

- to provide the styles for multiple HTML pages & prevent repetition of code across those files.

```
<link rel='stylesheet' href='style.css' />
```

(for linking the stylesheet)

- * To add or give more preference we need to give the styles after the ~~to~~ link tag of stylesheets.

As it's interpreted language so the priority will be given to the last style that is being applied.

- * Selectors which select an element/tag are called Element Selector

```
p, div {  
  color: red;  
}
```

--> we can have multiple element selector

- * Class as an attribute is used to map elements to their styles in CSS.

Class
Selector

→ .p1 {
 color: blue;
}

<p class="p1"> Hi </p>

- * Class is used to identify a group of elements.

- * Id is used to identify a ~~group~~ particular element uniquely.

Id
Selector

→ #div1 {
 color: green;
}

- * We can use the type of the input tag to select the tag. i.e.
We can use the attribute to select the element

Attribute
Selector

→. [type = 'submit'] {
color: green;
}

[type = 'reset'] {
color: red;
}

- * to select a child-element inside a parent element, we use descendant Selector

```
div p {  
color: red;  
}
```

- * to select the direct child^{elements} only we use child Selector

```
div > p {  
color: red;  
}
```

- * Pseudo Selectors :- to apply some css on elements, when an action/event occurs.

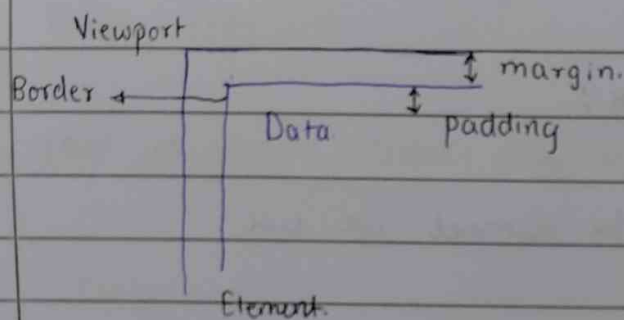
```
div : hover {  
color: yellow;  
}
```

```
div :nth-child (even) {  
color: green;  
}
```

* Universal Selector - to apply for all elements within the body.

```
* {  
  font-family: monospace;  
}
```

* Box - Model



Every div has a:-
i) margin
ii) border
iii) Padding

border: 1px solid red;
(size) / (style) (color)
(width)

* margin:- Space between border and viewport / another element;

* padding:- Space between the border and the content inside the box;

* margin: T R B L;
(px) (px) (px) (px)

* Every element in HTML is considered a rectangular box;

* Display (property)

* For `display: inline`, the width and height are ignored (for

`display: inline;`

* but, to consider the width & height in inline, we use

`display: inline-block;`

* to not show/display the element we use

`display: none;`

* position, to adjust based on the viewport

- by default position is static, so it ignores the top, left, right, bottom properties.

(considers)

- to take in account the properties (top, left, right, bottom) we use `position: relative;`

↳ arrange the elements relative to the present/existing elements.

- `position: absolute;`

↳ this will make sure the elements are arranged absolute to the viewport. So even if a element is present there it'll overlap the previous element.

* position: fixed;

- So, this will fix the position of element on the screen/viewport even if we scroll up/down.

* Float (property)

Float: left; → will do position = ~~left~~ & display = inline-block;

(So, we should declare only one either float or display)

* flex

* Generally represents a 1-dimensional display

* [to use flex, we need to use it inside a container]

&

- the container on which flex is applied is called flex-container
- the elements inside that container are called flex-items.
- to set the direction of elements we use flex-direction.

flex-direction: column; (default)

flex-direction: row; (arranges ~~row~~ elements in row)

flex-direction: column-reverse; (arranges elements in col. in reverse)

flex-direction: row-reverse;

- to justify the content ⁱⁿ the direction of flex, we use justify-content

justify-content: center;

justify-content: space-around;

justify-content: space-between;

justify-content: space-evenly;

* grid

(represents a 2-Dimensional display)

(elements in grid are called grid-items)

display: grid;

grid-template-columns: 100px 100px;
(1st) (2nd) → columns

grid-template-rows: 100px;
↳ size.

(for a row we cannot give the no. of rows, we can just give the number of columns)

(we can give the size of columns in pixels or in fractions)

grid-template-columns: repeat(5, 1fr); (5, 1fr)

↓

will create 5 cols with 1 fr size

* border-radius: 5px; → to make border round

* text-shadow: 10px 0px gray;
x-axis y-axis color

or

: 10px 0px 5px gray;
x-axis y-axis blur-radius color

box-shadow: 10px 0px 5px black;
x-axis y-axis blur-radius color

or

10px 0px 5px 10px black;
x-axis y-axis blur-radius spread-out color

*/ gradient

background : radial-gradient (red, blue)

- starts red at centre & spreads outward with blue.

or

: linear-gradient (red, orange)

- starts ~~for~~ red from top and gradually blends in orange at bottom.

*/ Animations :-

div1: hover {

transform: rotate (360 deg); --> rotates 360 degrees.

transition: 2s; --> transition period of 2 seconds

}

div2: hover {

transform: scale (1.5) --> will scale the element 1.5x of its size.

transform: translate (100px, 100px); --> move div 100px in x & y axis
(x) (y)

*/ Column-count

p {

column-count: 2;

}

PTO

* Media - Query

for responsive design on mobile,

```
@media screen and (min-width: 720px) {
```

```
  #h1 {
```

```
    display: block;
```

```
  }
```

```
  #h2 {
```

```
    display: none;
```

```
  }
```

```
background-color
```

```
  body {
```

```
    background-color: beige;
```

```
  }
```

```
}
```

--- on mobile ---

```
@media screen and (min-width: 100px) {
```

```
  #h1 {
```

```
    display: none;
```

```
  }
```

```
  #h2 {
```

```
    display: block;
```

```
  }
```

```
  body {
```

```
    background-color: green;
```

```
  }
```

```
}
```

↳ properties applied when width [100, 720) px.

* Breakpoints (class Infix)

Breakpoint	Class Infix	Dimensions	
Extra-small	None	< 576 px	(0 - 575 px)
Small	sm	≥ 576 px	(576 - 767 px)
Medium	md	≥ 768 px	(768 - 991 px)
Large	lg	≥ 992 px	(992 - 1199 px)
Extra-large	xl	≥ 1200 px	(1200 - 1399 px)
Extra extra large	xxl	≥ 1400 px	(1400 - above px)

* `.container - { breakpoint }` :- for all the breakpoint below the specified breakpoint it'll take width 100% and above the breakpoint, it'll follow the above table.

* `.container-fluid` :- for all the breakpoint, it'll consider the width as 100%.

* `class = "col"` :- treats the div as a column.

`class = "row"` :- treats the div & inside elements as ⁱⁿ a row.

* By default the viewport is divided into 12 columns, so we can specify different width of each columns, i.e.

`class = 'col-6'` --> This will make the column of size of 6 column divisions

So, if no. of columns exceed the total 12, it'll bring the columns on new line/below the current row.

* Typography

* we can use different header classes for elements.

```
<p class="h1"> Hello World </p>
```

↳ {1,2,3,4,5,6}

* for display headings

```
<p class="display-1"> Hi there </p>
```

↳ {1,2,3,4,5,6}

— / — / —

- document, console, window are objects by default present in browser.

* Internal JS:- JS written inside the script tag in a HTML file.

* External JS :- JS written in a JS file & importing the file in the HTML file.

* Var is deprecated (so don't use);
(use let or const instead)

console.log("num1" + 10) $\rightarrow$ "num1 10"

Concatenation

* Different datatypes of JavaScript :-

2) Number let $n = 10$;
 let $n = 11.25$; } both are Number.

2) Boolean `let n = true;`
 `n = false;` } boolean datatype.

3) String

let n = 's'

let n = "Sun"

} both are String datatype.

Special Values

* undefined (Special datatype & a value undefined)

let n;

↓

→

Value = undefined

datatype = undefined

* null

let n = null;

↓

→

Value = null

datatype = Object

* let result = 10/0

↓

→

Value = Infinity

datatype = number

* let result = "Sunbeam" * 10

↓

→

Value = NaN

datatype = number

Datatypes

Values

Variables

- Number

- undefined

- let

- String

- NaN

- Const

- Object

- NULL

- Boolean

- Infinity

- Undefined

- function

___/___/___

* Function in JS

```
function f1 ()  
{  
  console.log("Inside f1");  
}
```

* <script>

Not Recommended $n3 = 10;$ ---> with ~~no~~ variable keyword, such variables are called Global variables

```
function f1 ()  
{  
  console.log(n3); // 10  
}  
  
</script>
```

* To avoid such behaviour, we use the stricter version, ^{by} ~~called~~ writing "use strict" at top of file.

* Parameter Passing

```
function f1 (n1)  
{  
  console.log(n1 + " " + typeof(n1));  
}  
  
f1(10);
```

* Question

```
*
function f1 (n1)
{
    console.log (n1);
}

function f1 (n1, n2)
{
    console.log (n1, n2);
}
```

```
f1 () ;
f1 (10) ;
f1 (10, 20);
```

} all 3 will call the f1(n1, n2) function.
as the js is a interpreted language so
the JS engine will replace the entire
implementation of f1 with latest
implementation code.

Imp

remember *

* null * null = 0

* true * true = 1

* "sun" * "sun" = NaN

* undefined * undefined

```
* function square (n1)
{
```

```
    const sqr = n1 * n1;
    return sqr;
}
```

```
function square (n1)
{
```

```
    console.log (n1 * n1);
}
```

```
square (5);
```

```
let result = square (7);
```

```
console.log ("res is -" + result);
```

output is - 25

49

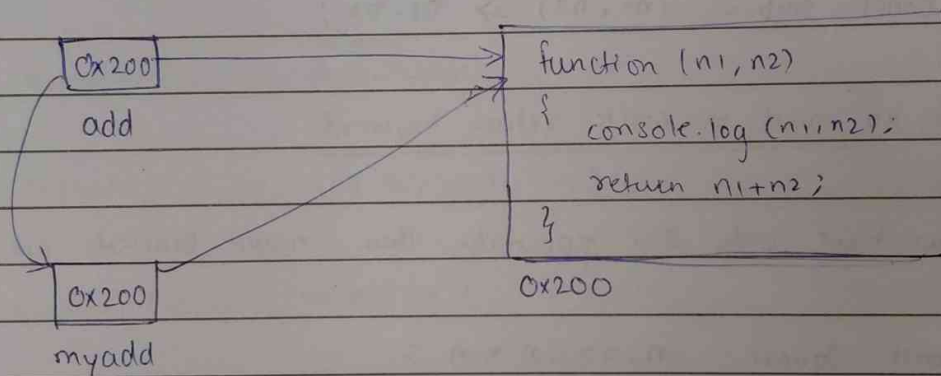
as the latest square
function doesn't return + ----- res is - undefined.

```
*
function add (n1, n2)
{
  console.log (n1, n2);
  return n1+n2;
}
```

```
add(10, 20);
console.log ("value is " + add); // displays the whole fun" definition
console.log ("type of " + typeof(add)); // type of function.
```

let myadd = add --> function alias

* Function Storage



* Anonymous Function

A function does not having a name is called as a Anonymous Function.

```
const myAdd = function (n1, n2) {
  console.log (n1+n2);
}
```

* Arrow Function

```
const myAdd = (n1, n2) => {  
    console.log(n1+n2);  
}
```

```
const sub = (n1, n2) => {  
    return n1 - n2;  
}
```

```
const res = sub(10, 5); // 5
```

* If only one statement in function, using "{}" is optional.

```
const sub = (n1, n2) => n1 - n2;
```

also, no need to write return keyword.

* If we have only one parameter then round brackets are optional.

```
const square = n => n * n;
```

* Higher Order Function

```
function executor (n1, n2, fn)  
{
```

```
    return fn(n1, n2);  
}
```

```
executor(10, 20, add);
```

```
function add(n1, n2)  
    return n1 + n2;
```

```
function sub(n1, n2)  
    return n1 - n2;
```

```
function mul(n1, n2)  
    return n1 * n2;
```

* Functions like `executer`, which accepts another function, Such functions are called as Higher Order Function.

* Functions like `add`, `sub`, `mul`, which are passed to higher order functions as an argument are called as Callback Function.

* Other ways to write

```
→ function executer (n1, n2, fn)
{
    const res = fn (n1, n2);
    console.log (res);
}
```

~~`executer (10, 20, (n1, n2) => n1 + n2)`~~

→ `executer (10, 20, (n1, n2) => n1 + n2)`

OR

```
→ executer (10, 20, (n1, n2) => {
    const res = n1 + n2;
    return res;
})
```

* Objects

3 ways to create a object in JS

i) using `{}`

ii) using `Object`

iii) Using Constructor funⁿ

in ~~C++~~ → in JS
* data members → Properties
* member functions → methods

* In JS, we can first create an object and then add the properties to the object.

1)

```
let p1 = {};  
p1.name = "Vaibhav";  
p1.age = 23
```

OR

```
const p2 = {}  
p2.name = "Om";  
p2.age = 23;
```

} Possible / Allowed.

here the address of p2 is constant, not the contents inside it.
so, we can't do this,

```
p2 = {}
```

 ---> redeclaration not allowed

2) Another way

```
let p1 = {};  
p1["name"] = "Vaibhav";  
p1['age'] = 23;
```

```
const p2 = {};  
p2['name'] = "Om";  
p2['age'] = 23
```

OR

```
const p3 = {  
  name: "Vedika",  
  age: 22,  
}
```

} JSON objects

```
p2.age = 29;
```

// if value exists, it'll replace it otherwise it'll create that property

* Another way

```
const p1 = new Object(); (in Heap)
```

```
p1.name = 'omkar'
```

```
p1.age = 24
```

3) Using Constructor Function

```
function Person () {}
```

```
const p1 = new Person ()
```

OR

```
function Person (name, age) {
```

```
  this.name = name;
```

```
  this.age = age;
```

```
}
```

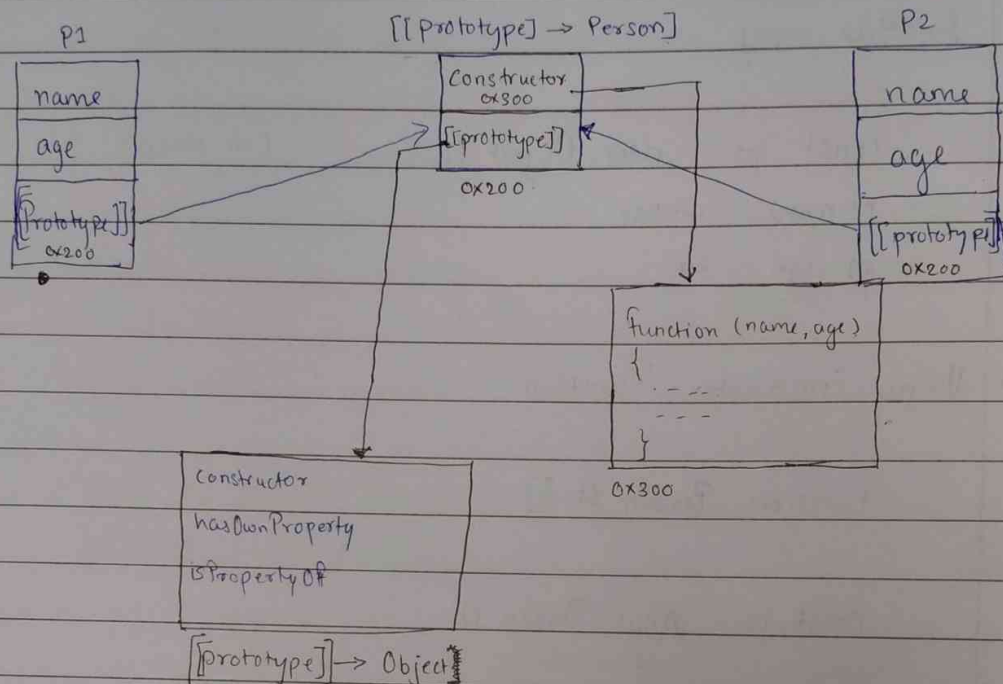
```
const p2 = new Person ("maki", 22);
```

```
const p3 = new Person (); // name & age are undefined.
```

this will execute.

* Every Object in JavaScript is derived from another Object called as Object

* Every Object is having a prototype object inside it.



* to add a function to the object

`p1.displayPerson = displayPerson;`

* to add this property to every single object, we add it to the prototype.

* `Person.prototype.displayPerson = displayPerson;`

(Incorrect way)

```

function Person (name, age)
{
  this.displayPerson = displayPerson;
}
  
```

This will create separate copy of method in each object.

* Hierarchy

- Reusability

1) Association

2) Inheritance

→ If only one instance of an ^{class} ~~object~~ inside a class, we use Inheritance.
another

→ If we can have multiple instances of a class, then we use Association.
~~multiple instance~~

* In JS, every block of memory is considered as an Object.

* To initialize/ the employee object from Person class
derive

```
function Employee (id, name, age, salary) {
```

```
    Person.call (this, name, age); // Just like constructor
```

```
    this.id = id // delegation in c++;
```

```
    this.salary = salary
```

```
}
```

But here the prototype of the Parent (Person) class is not inherited/ present in the employee's prototype so, some methods like (displayPerson) are missing.

* Way 1 for prototype inheritance

Employee.prototype = Person.prototype

(But, this will replace the employee's prototype with the person's prototype, so, that will contain only the methods of person prototype, not the methods of employee)

* Way 2 for prototype inheritance (for testing purposes only)

Employee.prototype.__proto__ = Person.prototype

(Here methods & prototype of both will be present but, this is not recommended)

* Way 3 for prototype inheritance

Employee.prototype = Object.create(Person.prototype)

(This will also do the job, but we'll have to assign the methods once again in the prototype manually)

* Way 4 (correct way) for prototype inheritance

Object.setPrototypeOf(Employee.prototype, Person.prototype);

(This will perform the job correctly by inheriting all the methods of person ~~proto~~ class)

* Class

```
class Date {
```

```
    constructor (day, month, year)
    {
```

```
        this.day = day
```

```
        this.month = month
```

```
        this.year = year
```

```
    }
```

```
    displaydate ()
```

```
    {
```

```
        - - - - -
```

```
        - - - - -
```

```
    }
```

```
}
```

```
class Employee {
```

```
    constructor (id, name, salary, day, month, year)
```

```
    {
```

```
        this.id = id
```

```
        this.name = name;
```

```
        this.Salary = salary;
```

```
        this.doj = new Date (day, month, year)
```

```
    }
```

```
    displayEmployee ()
```

```
    {
```

```
        - - - - -
```

```
    }
```

```
}
```

* class Person {

constructor () {

}

display () {

}

}

class Employee {

constructor () {

}

display {

}

}

* function Person (---)

{

}

function Employee (---) {

}

Person.prototype.display = function ()

{

}

Employee.prototype.display = function ()

{

Object.getPrototypeOf(Employee.prototype)
.display.call(this);

} OR

// Person.prototype.display.call(this)

→ This should be avoided and
it's wrong as it may happen the
display method might not be defined
& it'll be undefined.

const e1 = new Employee (---);

e1.display() // this will call the method of employee

* Using Class

```
class Person {
```

```
    constructor ()
```

```
{
```

```
}
```

```
    display Person ()
```

```
{
```

```
}
```

```
}
```

```
class Employee extends Person {
```

```
    constructor () {
```

```
}
```

constructor
delegation

(will call the
constructor of
parent class)

~~~~~

~~Person~~ ~~constructor~~

Super (----);

```
display () {
```

```
    Super.display ()
```



will call the display fn of  
Person (parent) class.

\* Super (----) Should be the first statement in the constructor body.

```
class Emp extends Person {
```

```
    constructor (id, name, salary, mobile)
```

```
{    Super (name, mobile);
```

→ 1<sup>st</sup> instruction/statement.

```
    ----
```

```
    ----
```

```
}
```

\* Super with "(")" is called a Super Statement and is only used in constructor body.

\* Super with "." is called a Super keyword.

\* In JS, multiple class inheritance is not supported.

## \* Array in JS

- Array as a DS provides us index based operations.

① `const arr = [];`                      type: Object

OR

② `const arr2 = new Array();`      type: Object

Contains a hidden property  $\rightarrow$  prototype (consisting all the methods which can be called on an array object)

\* Push :- to push an element in array

`arr.push(60);`  $\rightarrow$  will be pushed at last index

\* forof loop :-

(for-of)

`for (e of arr)`

`console.log(e);`      // 10 20 30 40 50

\* for-in loop :-

`for (e in arr)`

`console.log(e);`      // 1 2 3 4 5

(returns the indexes)

$\rightarrow$  used for printing the properties of objects

`const p1 = {`

`name: "om",`

`age: 23,`

`for (e in p1)`

`console.log(p1[e])`  $\rightarrow$  this'll print

## \* ForEach

```
arr.forEach ( (e) => console.log(e) )
```

callback function

## \* Filter

```
arr.filter (e => e % 2 == 0).forEach (e => console.log(e));
```

(Such Expressions are called **lambda expression**)

## \* map

```
const res = arr.map (e => e * e);
```

arr of  
↑ square of each element.  
in arr

(map can return anything unlike filter which returns only a number)

(forEach doesn't return anything)

## \* Template Strings

```
arr.forEach (e => console.log(`name: ${e.name}, age: ${e.age}`));
```

## \* Hidden argument

\* A function in JS can be passed any no. of arguments.

```
function f1 ()  
{  
  console.log (Arguments)  
}
```

→ array of arguments which is kept hidden

f1 ();

f1 (10);

f1 (10, 20);

f1 (10, 20, 30);

\* Rest Operator — encapsulates and create an array by grouping them together

```
function f1 (...arr)  
{  
  console.log ("f1");  
  for (e in arr)  
    console.log (e);  
}
```

if ...arr before var-name then  
it's rest operator.

or left hand side of "=" operator.

f1 (10, 20, 30, 40);

→ When call happens. (...arr) will group  
them into an array (arr)

PTO

## \* Spread Operator

→ used to spread out the properties into a new object.

(if we use (...) on RHS of "=" operator, then it becomes a spread operator)

```
const p1 = {  
  name: "Vaibhav",  
  age: 23  
}
```

\* `const p2 = {... p1}`

```
console.log(p2) // { name: "Vaibhav", age: 23 }
```

or

\* `const p2 = {... p1, age: 19}`

```
console.log(p2) // { name: "Vaibhav", age: 19 }
```

\* `const p2 = {... p1, age: 18, mobile: 9370065087}`

```
console.log(p2) // { name: "Vaibhav", age: 18, mobile: 9370065087 }
```

\* `courses1 = [...], courses2 = [...]`

```
all courses = [... courses1, ... courses2]
```

## \* DOM (Document Object Model)

- it is a tree consisting of Nodes & Elements
- it is stored inside the window property called as Document.

\* to access it as a JS object, we do

~~console.dir(document)~~

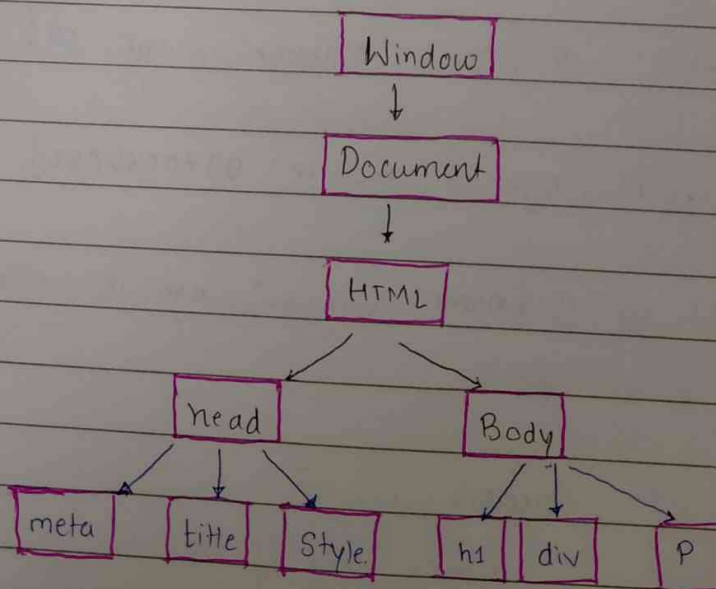
> console.dir(document)

> document.children --> returns the HTMLDocument consisting of only one child, that is the HTML (root)

> [HTMLDocument]: HTML[1] tag of our HTML page.

> document.children[0] --> will return the entire HTML of the page.

> document.children[0].children --> will return the array of direct childrens (i.e. [head, body])



\* To get the root node of Dom Tree, we use

> `document.firstElementChild` --> will return the array of all childrens of root element.

\* To get the last child of ~~the~~ a tree, we use

> `document.lastElementChild`

\* To get ~~get~~ Element based on Id.

> `document.getElementById("id-p")` --> returns element

## jQuery

### \* Selector

```
const h1 = $('h1') // element Selector
```

```
const h1 = $('.h1') // class Selector
```

```
const h1 = $('#h1') // id Selector
```

### \* to set the inner text of the tag/element

```
h1.text = ".....";
```

OR

```
$('#id-h1').text = ".....";
```

### \* val() vs value

\* In jQuery, value is just an attribute of element, which provides the value of the element.

\* While, val() is an function property of the object returned by the jQuery, so, it can be called to fetch the value of an element.

\* to add a row to the element in jQuery, we use

```
$('#tbody').append(row)
```

\* to hide a matched element, we use

`$('#div1').hide()` --> will hide the div

\* to show a matched element, we use

`$('#div1').show()` ---> shows hidden element

\* **AJAX (Asynchronous Javascript and XML)**

\* To send a request to server we use **XMLHttpRequest**

```
const xhr = new XMLHttpRequest();
```

```
// configure xhr object for sending the request
```

```
xhr.open("GET", "http://127.0.0.1:5500/employees.json");
```

```
// Send the request to server
```

```
xhr.send();
```

```
// when server returns a response, it calls a method
```

```
xhr.onreadystatechange = () => {
```

```
    console.log("Ready State changed");
```

```
}
```

→ when server returns response, it returns us 2 main things, the readyState and the status code.

\* There are 4 readyState

- 1) 1 → xhr object is configured
- 2) 2 → request is received by server
- 3) 3 → server is processing the request
- 4) 4 → server has completed/finished the processing of request.  
(The data is available when readyState is 4)

\* So, to print the data

```
xhr.onreadystatechange = () => {  
  console.log('Ready State' + xhr.readyState)  
  console.log('Status Code' + xhr.status)  
}
```

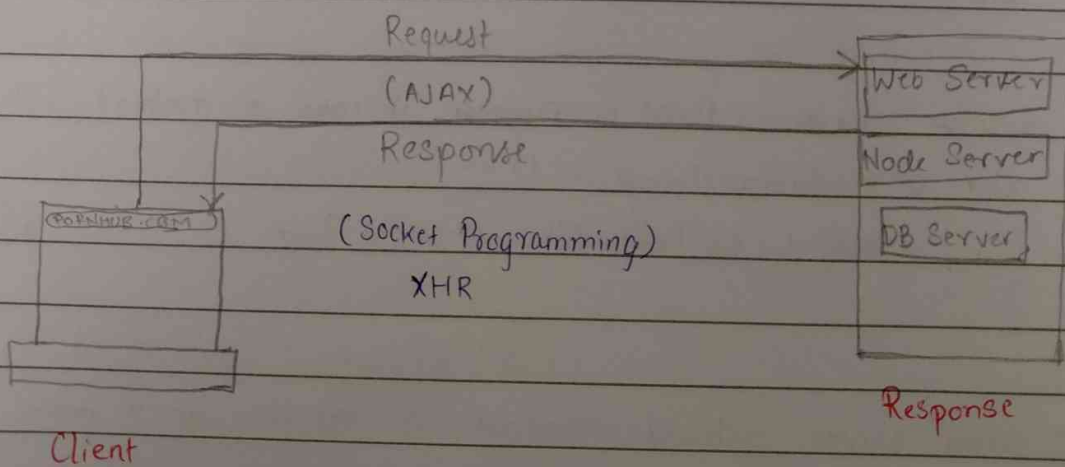
```
if (xhr.readyState == 4 && xhr.status == 200)
```

```
  const res = xhr.response; // data is given in the form  
                             of string
```

```
  const emps = JSON.parse(res);
```

```
  console.log(emps) // JSON object
```

```
}
```



## AJAX using jQuery

```
* function getEmployees () {  
    $.ajax ({  
        method : "GET",  
        url : "http://127.0.0.1:5500/employees.json",  
        success : (response) => {  
            const emps = response; // This is a JSON object  
            // do the display of employees.  
            -----  
            -----  
        }  
    });  
}
```

## Promises

// Written By Senior Dev

```
internal library JS) function getEmployees (fn) {  
  
    // Perform xhr & fetch emps data  
    if (emps)  
        fn (emps, null);  
    else  
        fn (null, {error: "not found"});  
}
```

// Written By Junior Dev

```
function getAllEmployees ($) {  
  
    getEmployees ((data, err) => {  
        if (data)  
            console.log (data);  
        else  
            console.log (err);  
    })  
}
```

- \* Promises were introduced in order to deal with the callback hell.

Promisified Version of Previous code:-

```
function getEmployees () {  
  // perform xhr & fetch data  
  
  const promise = new Promise ((resolve, reject) => {  
    if (emps)  
      resolve ({emps, error: "not found"});  
    else  
      reject ({null, error: "Something wrong"});  
  });  
  
  return promise;  
}
```

- \* Promise constructor function's <sup>callback fn.</sup> takes in 2 parameters these are both  $() \Rightarrow \{ \}$  arrow functions (resolve & reject)
- \* The resolve & reject only accepts 1 parameter (value), so, we pass a Single Object
- \* If promise is fetched, then it can be either
- i) Fulfilled
  - ii) rejected
  - iii) pending state.
- \* If promise is fulfilled, then we can use the  $\text{.then}$  method to fulfill it.

```
function getAllEmployees ()
{
```

```
  getEmployees ()
```

```
    .then ( (data) => {
```

```
      console.log (data);
```

```
    })
```

```
    .catch ( (error) => {
```

```
      console.log (error);
```

```
    })
```

\* IF ~~we~~ the promise is rejected, then we need to handle the promise using the (.catch) method and process the error.

\* If we have nested function which returns promises, it can be done like,

```
function getAllEmployees ()
{
```

```
  getEmployees ()
```

```
    .then ( (data) => {
```

```
      console.log (data)
```

returned promise is  
resolved by this  
.then

```
      return getDepartments ()
```

```
    }
```

```
    .then ( (data) => {
```

```
      console.log (data);
```

```
    }
```

universal  
catch

```
    .catch ( (err) => {
```

```
      console.log (err);
```

```
    });
```

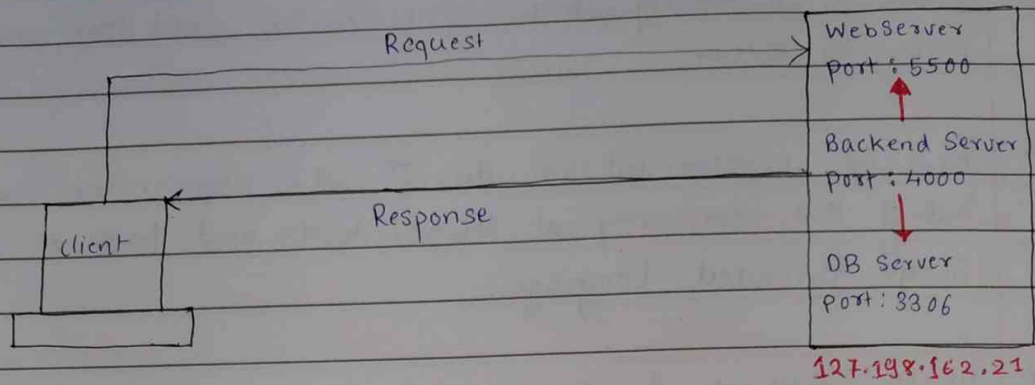
## Async Await

```
async function getAllEmployees () {  
  try {  
    const emps = await getEmployees ();  
    console.log(emps);  
    const depts = await getDepartments ();  
    console.log(depts);  
  }  
  catch (err)  
    console.log(err);  
}
```

## Fetch

```
function getEmployee ()  
{  
  const p = fetch ('http://127.0.0.1:5500 /employees.json')  
    .then ((response) => {  
      return response.json();  
    })  
    .then ((data) => {  
      console.log(data);  
    })  
    .catch ((err) => {  
      console.log(err);  
    })  
}
```

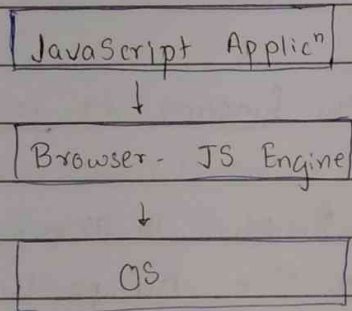
## Node JS



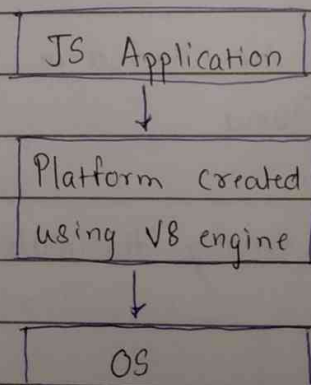
D:/ecommerce

- \* Backend Server maintains the connection between the Web Server and DB Server

- login.html
- employees.json



- \* To run JS applications, directly on the runtime environment a platform was created called as Node.

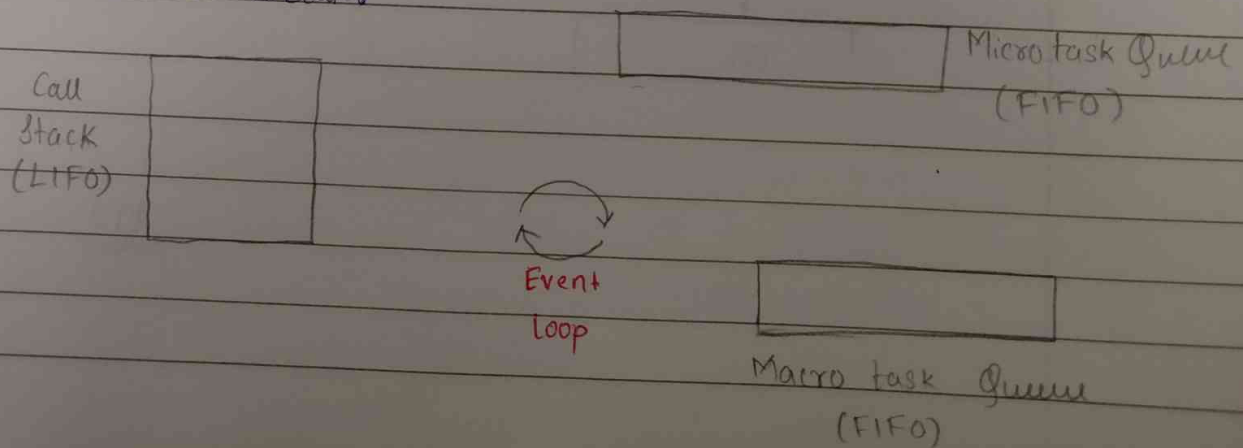


PTO

- \* Libuv is a library which implements the event loop inside it, written in C/C++.
- \* Runtime environment provides JS, it's asynchronous nature by hiding the complexity at lowest level and lets JS remain a single-threaded language.
- \* All the callback functions, wait in a callback Queue waiting for the call stack to be empty, Once it's empty event loop will then sequentially all the functions ~~is~~ present in Queue.
- \* Call Stack
- \* a data structure which holds the function it's going to execute
- \* Call Stack is LIFO, so, if a function is present in stack & a new function is pushed in then, the pointer will move to new function and will start executing that function
- \* Call Stack Queue

All the call-back functions will be stored in a Queue Data Structure called as Call Stack Queue.

So, once the call stack becomes empty, the callback functions will then be executed



\* Functions with highest priority, their callback functions will be pushed on Micro-task Queue (Promises, etc)

\* Functions with lowest priority, their callback functions will be pushed on Macro-task Queue (setTimeout, setInterval, etc)

For example:-

```
console.log("Start");
```

```
setTimeout(() => {  
  console.log("Inside Timeout");  
}, 0)
```

// priority is low, so callback is pushed to macro task queue

```
Promise.resolve().then(() => {  
  console.log("Inside Promise");  
});
```

// Priority is high so callback is pushed to micro task queue.

```
console.log("Finish");
```

(Imp) \* Output:-

\* Execution :-

Start

i) console.log will be executed.

~~Finish~~ Finish

ii) setTimeout will be called & a callback will be pushed to macro queue.

Inside Promise

Inside setTimeout

iii) Promise.resolve() will get executed & after .then() will be called and a callback will get pushed to micro queue.

iv) console.log will be executed.

v) Now event loop will check in micro queue & execute those callback functions.

vi) And after that event loop will execute the macro queue, callback functions.

\* Functions with highest priority, their callback functions will be pushed on Micro-task Queue (Promises, etc)

\* Functions with lowest priority, their callback functions will be pushed on Macro-task Queue (setTimeout, setInterval, etc)

For example:-

```
console.log("Start");
```

```
setTimeout(() => {  
  console.log("Inside Timeout");  
}, 0)
```

// priority is low, so callback is pushed to macro task queue

```
Promise.resolve().then(() => {  
  console.log("Inside Promise");  
});
```

// Priority is high so callback is pushed to micro task queue.

```
console.log("finish");
```

(Imp) \* Output:-

\* Execution :-

Start

i) console.log will be executed.

~~End~~ Finish

ii) setTimeout will be called & a callback will be pushed to macro queue.

Inside Promise

Inside setTimeout

iii) Promise.resolve() will get executed & after .then() will be called and a callback will get pushed to micro queue.

iv) console.log will be executed.

v) Now event loop will check in micro queue & execute those callback functions.

vi) And after that event loop will execute the macro queue, callback functions.

\* In JS, normal functions (function abc() {} ) are directly pushed into call stack, whereas arrow functions (() => {} ) are not directly pushed to call stack but to the queue based on the priority.

\* For every function pushed into call stack a Function Activation Record is created. (FAR)

### \* Modules

\* For exporting :-

```
module.exports = {  
    add,  
    sub,  
    mul  
}
```

\* for importing :-

```
const math = require('./math.js');
```

\* If we export just,

```
module.exports = add --> This will export as a function itself.
```

So, when importing, we just provide a suitable alias.

```
const add = require('./math.js');
```

\* `module.exports` is internally an object, so when we do,

`module.exports = add`, it gets replaced by function definition.

\* we can have multiple `module.exports`, but it'll be replaced by the latest `module.exports`, i.e.

`module.exports = add`

`module.exports = sub`

So, when importing, will get `sub`, not `add`.

```
const math = require('./math.js');
```

↓

(Sub: Function)

\* We can add properties to `module.exports`, by using

`module.exports.add = add`

`module.exports.sub = sub`

`module.exports.mul = multiplication`

So, this will be imported as

```
const math = require('./math.js');
```

↓

```
{ add: add, sub: sub, mul: multiplication }
```

↓

function

(Imp) \* In nodejs, functions are also treated as Objects, so, when exporting, we can add an object to the function.  
→ (property)

```
module.exports = add  
module.exports.Sub = sub
```

So, when importing `const math = require('./math.js');`

\* we can do, `math(10, 20)` --> This will evaluate as add function  
`add(10, 20)`

\* and to access the sub property we use,

`math.sub(20, 10)` --> This will evaluate the sub function

\* But the vice versa is not possible, so we can't do

```
module.exports.sub = sub  
module.exports = add
```

→ here we assign an object to `module.exports` & then we are assigning an function to an obj. which will result in error.

(The above code will give an error)

\* If exporting multiple `module.exports`, if we do,

```
module.exports = add  
module.exports = {sub, mul}
```

This will get overridden/replaced by the next/latest export. So, only `{sub, mul}` will be exported, not add.

(Vice versa is also possible, i.e. object will get replaced by function)

→ in modern JS

\* We can have only one default export & multiple exports, i.e.

```
export default add;
```

```
export { sub, multiplication };
```

So, when importing, we do,

This will have  
"add" function

```
import math from './math.js';
```

```
import { sub, multiplication as mul } from './math.js';
```

→ other exports

## \* HTTP Servers

\* `const http = require('http');`

~~const Server = http.createServer;~~

```
const Server = http.createServer((req, res) => {
```

```
  console.log(req.method);
```

```
  console.log(req.url);
```

```
  res.end("Hi there");
```

```
})
```

```
Server.listen(4000, "localhost", () => {
```

```
  console.log("Server started on 4000");
```

```
})
```

\*  
const http = require("http");

const server = http.createServer((req, res) => {

const method = req.method;

const url = req.url;

if (url === '/users')

res.end("user data");

else if (url === '/product')

res.end("product data");

else

res.end("use the correct API");

server.listen(4000, "localhost", () => {

console.log("Server started on 400");

});

## \* http backend Server

# routes

- \* It is a way to connect with backend server
- \* It helps to decide what task needs to be performed for the incoming request
- \* The route can be configured by identifying the method and the path from the incoming request

- \* Method - GET Path - /user : get all the users from the db
- \* Method - POST Path - /user : add the user in db
- \* Method - PUT Path - /user : update the user in db
- \* Method - Delete Path - /user : delete the user from db

- \* To write the contents in response first & then send we use,

```
res.write('<h1> Hello </h1>');  
res.write('<h2> World </h2>');  
res.end(" ");
```

- \* To set headers for response, we use

```
res.setHeader('content-type', 'application/json');
```

- \* If we use the method multiple times, then we ~~can~~ will get the header set to the latest/last header, that was being set, as it's a setter function.

PTO

\_/\_/\_

\* For every request sent for fetching resources/pages, a **Preflight Request** is sent to server, in order to check, if the source is allowing CORS / resource sharing from diff origin.

\* To restrict access to specific origin, to share the resource, we use  
Access-Control-Allow-Origin : ip-address or \* → (for everyone)

\* **Preflight Request**  
- Feature of Browser (Desktop)

|                             |                                   |
|-----------------------------|-----------------------------------|
| * Web Servers (live-server) | * Database Server (my-sql) (3306) |
| - Hosting                   | - Store-data                      |
| - Web Pages                 | (172.18.1.19.12)                  |
| (172.192.1.1) (5500)        |                                   |

\* Backend-Server (express) (191.12.13.0) (4500)

- Socket programming with db
- SELECT, INSERT, UPDATE, DELETE
- In return it gets JSON object.

\* We run all these 3 servers on 3 different devices, for better performance results.

\* `const app = express()` → This returns an express application

## \* Express with MySQL

\* we'll be using the mysql2 module

```
const express = require("express");  
const mysql = require("mysql2");
```

```
const pool = mysql.createPool({  
  host: 'localhost'  
  user: 'root'  
  password: 'root'  
  database: 'dac-db'  
});
```

```
app.get('/user', (req, res) => {  
  const sql = 'Select * from users';  
  pool.query(sql, (err, data) => {  
    if (data)  
      res.send({ users: data, status: "Success" });  
    else  
      res.send({ error: err, status: "Error" });  
  });  
});
```

## \* Router (in Express)

- it is used to create multiple routes in a modular way.
- it is also called as mini-express application.

```
const router = express.Router() --> (This is both a fn & obj)
```

```
router.get('/signup', (req, res) => { });
```

```
module.exports = { router }
```

## \* Middleware

- \* A function that executes for every incoming request
- \* Used to do some common checking across all routes.
- \* Can be used to get/set request or response headers (properties)

```
app.use ((req, res, next) => {  
  console.log ("Middleware");  
  next();  
})
```

```
app.use (router);
```

PTO

\_/\_/\_

(request)      having  
                         body  
                         ↑

\* When receiving a request from postman/frontend then we need to parse that request's body into a JSON object.

`app.use(express.json());`      // middleware to parse request's body.

## \* SQL Injection

```
const sql = `Select * from users where email = '${email}'`;
```

here sql injection can be performed, by passing a query inside the email value, by the use of string concatenation.

(for example:-

```
{
```

```
  "email" : "mukes@mail.com" OR '1' "
```

```
}
```

the single quotes will end a 1<sup>st</sup> arg/value and insert another query inside the ~~value~~ email value.

\* To avoid this, we write prepared queries.

```
pool.query(sqlquery, [email, password], (err, data) => {  
  -----  
});
```

```
const sql = `` Select * from users where email = ? and password = ?`;
```

(sql will be declared first, here it's just for example)

## \* JSON web Token

\* Token consists of 3 parts

- i) Headers
- ii) Payload
- iii) Signature

\* To check/authorize user to particular routes, we write our own middleware.

```
app.use ((req, res, next) => {
```

```
  const allowedURLs = ['/user/register', '/user/login']
```

```
  if (allowedURLs.includes(req.url))
```

```
    next();
```

```
  else {
```

```
    const btoken = req.headers.authorization.split(" ")[1];
```

```
    if (btoken)
```

```
    { const token = btoken.split(" ")[1];
```

```
      try {
```

```
        const payload = jwt.verify(payload, JWT_SECRET);
```

```
        req.uid = payload.uid;
```

```
        next();
```

```
      }  
      catch (err) {
```

```
      {
```

```
        res.send(result.createResult("Token is invalid");
```

```
      }  
    } else {
```

```
      resSend (result.createResult ("Token is missing"));
```

\* (In postman, we can use variables to set a URL for the whole collection & in each request it'll be applicable & we just need to write the variable name using "`{{url}}`"/user"     /    /      
(url = http://localhost:4000)

\* We need to verify if the user is deleted, then the token stored in cache or localStorage or sessionStorage, they need to be cleared.

\* To upload files to server (From frontend)

For this we use a module called as "`multer`".  
↑ key to identify the file.

`courseRouter.post("/", upload.single('courseImage'))`  
↑  
for single file upload

(upload is nothing but object of multer)

`const multerupload = require("multer");`

`const upload = multer({ dest: 'images/' });`

```
courseRouter.post("/", upload.single('courseImage'), (req, res) => {  
  const { name, description, fees } = req.body;  
  const { originalname, path } = req.file;  
  fs.rename(path, 'images/' + originalname, (err) => {});  
  const sql = 'Insert into courses (name, description, fees,  
    image values (?, ?, ?, ?)';
```

```
  pool.query(sql, [name, description, fees, originalname], (err, data)  
    {  
      if (data != '')  
        res.send(result.createResult(err, data));  
      else  
        res.send(result.createResult(err));  
    }  
  );  
});
```

\* When getting the images from server, we'll write a middleware which will set the destination of image.

→ This middleware needs to be added before `express.json()` middleware, so, it'll be

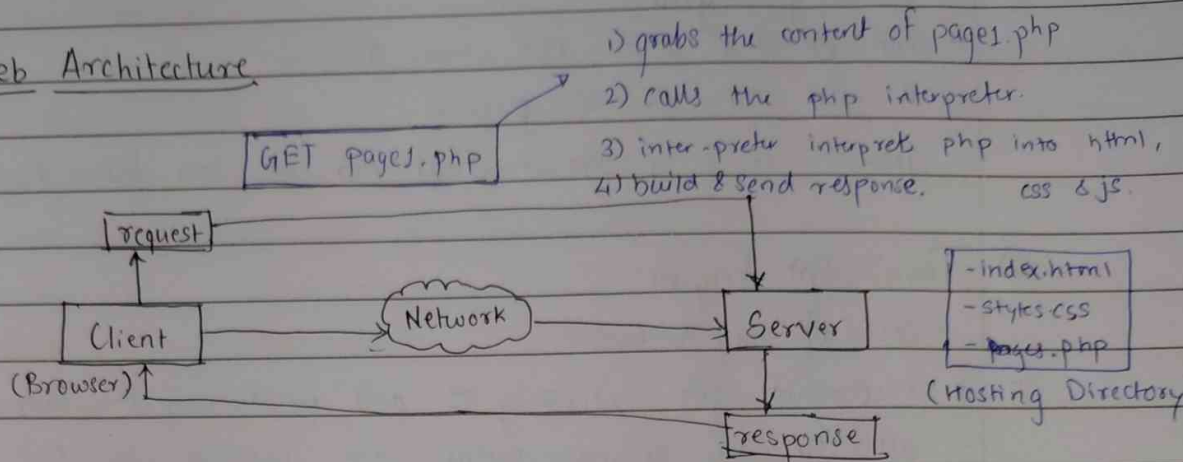
```
app.use('/images', express.static('images')) ;  
app.use(express.json());  
-----  
-----
```

\* To get the url of the page ~~on~~ from frontend using window obj., we use

```
window.location.href = 'courses.html' → To set url
```

# ReactJS

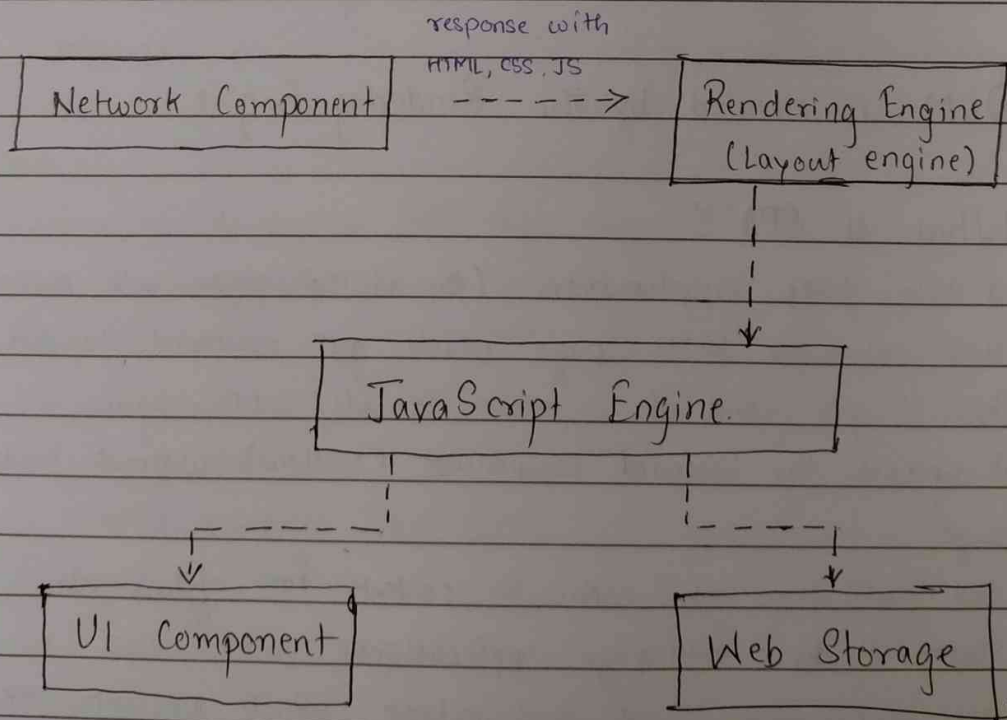
## \* Web Architecture



## Browser Architecture :-

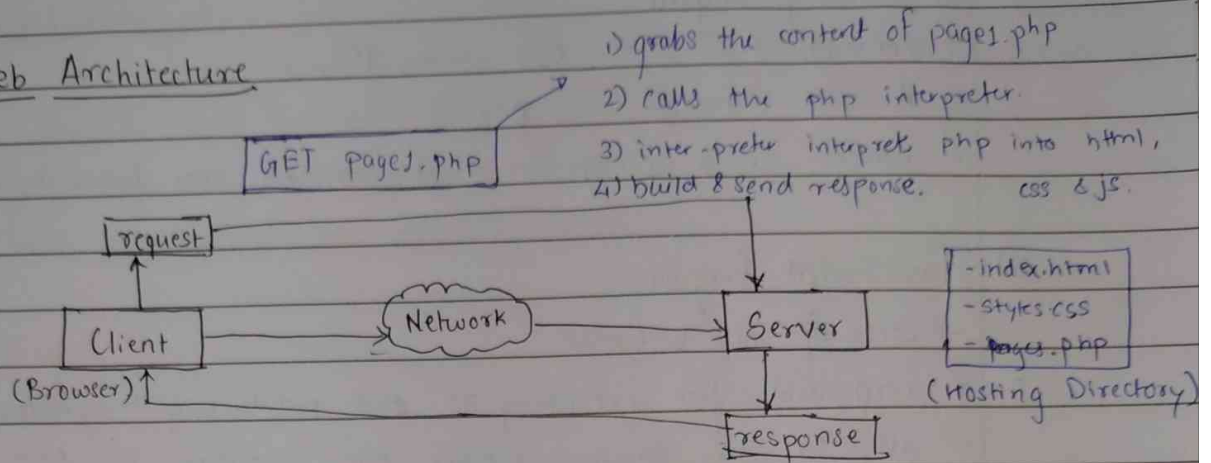
### ① Network Component :-

- responsible for sending request and receiving from server.
- communicates with server over network.



# ReactJS

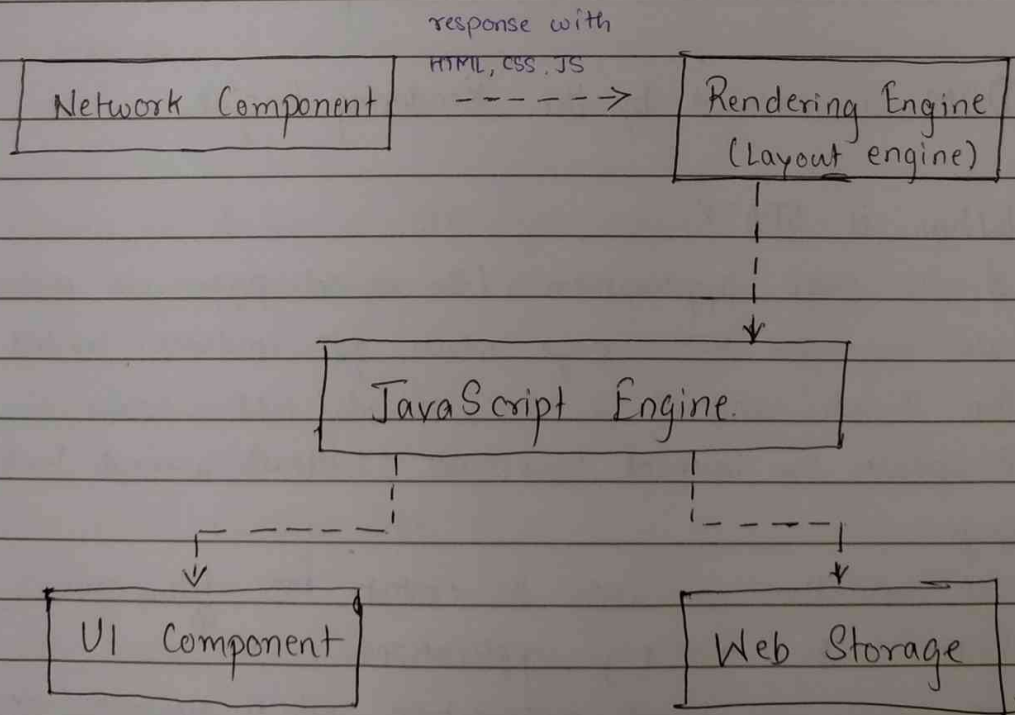
## \* Web Architecture



## Browser Architecture :-

### ① Network Component :-

- responsible for sending request and receiving from server.
- Communicates with server over network.



## 2) Rendering Engine

- converts the HTML, CSS  $\rightarrow$  JS
- it uses HTML & CSS parsers.
- parser is a program that converts one form to another form

## 3) JavaScript Engine

- core of any browser
- responsible for executing JS and producing the output.
- all the JS engines today are written in C++.

## 4) UI Component

- responsible for showing the output to the user on the viewport

## 5) Web Storage

- responsible for maintaining the history/sessions inside browser
- uses JS objects like `SessionStorage` and `LocalStorage`.

\* DOM is created by the Rendering Engine.

\* What is SPA?

- Single page application (the visited page gets cached)
- uses only one HTML page, which gets rendered/loaded initially.
- Once it gets rendered, it never reloads/loads again.
- \* = it updates the internal components (sections) without loading the entire page.
- \* - SPA application can cache the contents for offline use.
- faster than multi-page applications
- \* - uses component-based architecture, which promotes reusability.

### \* disadvantages

- the first time loading is bit slower than multi-page application
- it has to load all the components (sections) while loading the application for the first time.
- these applications are memory heavy

### - tooling

#### - language

- react (JS), angular (TS), preact (JS), Svelte (JS)

↓

library

↓

Framework

#### - bundler

- Tool to create a bundle / archive of all the required pages of web application)  
e.g. Webpack, vite

#### - Platform

- node.js

### \* React

- library to develop a SPA type application
- uses a component based architecture.
- free to use
- can be used
  - using CDN links
  - using bundlers like vite
- developed in JS.
- bit faster than angular.
- Since it is a library, it requires less memory compared to angular.

## \* Angular

- a framework used to develop SPAs
- TS is required to develop angular application
- bit slower than react as it requires more memory
- (react.development.js)
  - used to add
- (react-dom.development.js)
  - used to add virtual dom

\* `const h1 = React.createElement('h1', {}, 'welcome to react')`

param1 :- type of element to be created

param2 :- properties of element

param3 :- content of the element (children)

\* `ReactDOM.createRoot(root).render(h1)` // renders the h1 element created by react

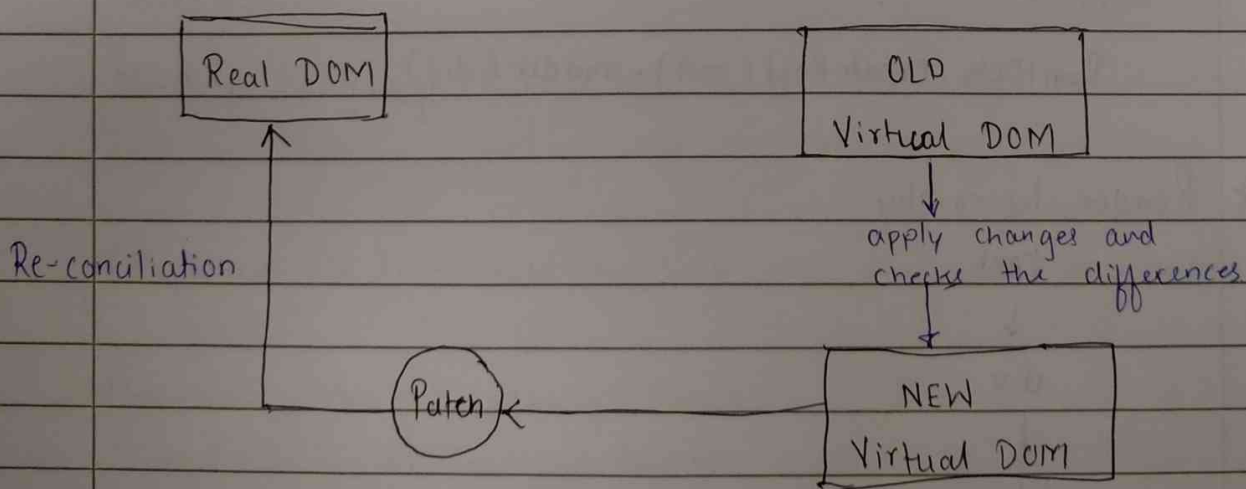
- Properties can be used/assigned by

```
{  
  className: 'red-element'  
}
```

## \* What is ReactDOM or virtual DOM?

- ~~in~~ in memory copy of Browser or real DOM
- real DOM is created by the browser to render UI in the browser's window
- when real DOM changes, the entire real DOM gets re-rendered
  - this is a very memory-heavy operation
  - this may slow down the UI generation

- react creates a in-memory representation of real-DOM called as virtual DOM, to solve the re-rendering of web-page, when there is a change in the web page.
- how virtual DOM works
  - when react application loads, react library creates a copy of real DOM and renders the entire page when it gets loaded initially (only for the first time).
  - when there is a change required, the change will be performed on the virtual DOM first
  - it will update the virtual DOM with new copy of changed element
- ★ - then react finds the difference b/w old DOM object and the new DOM object (diffing Algorithm)
  - then react creates a patch of new changes to be applied on the real DOM.
- ★ - finally the patch will be applied on real DOM to change the browser UI (reconciliation)
  - because of this process, the entire real DOM never reloads which is why the react application is faster than MPA.



|             |
|-------------|
| * Framework |
|-------------|

- generally developed using multiple languages like angular (TS+ JS)

|           |  |
|-----------|--|
| * Library |  |
|-----------|--|

- Single file developed using only one language and solves only one problem
- library requires less memory than a framework

- \* When having multiple elements, we can pass an array of elements to get rendered

React.  
DOM

```
createRoot(root).render([h1, h2, h3]);
```

- \* Rendering elements having parent-child relationship.

`const div = React.createElement('div', {}, h1);`

Parent ←                      → child

ReactDOM, createRoot (root). render (div)

- \* Render hierarchy

root

↓

div

↓

# h1

## \* What is Babel?

(compiler)

- Babel is a JS compiler, used to convert one form of JS to another form of JS.
- Generally used to convert higher version of JS to older version of JS, which the browser can understand.
- Babel is responsible for converting the JSX into `React.createElement` syntax.

### JSX Syntax

```
const h1 = <h1> Hi There </h1>
```

(lower version  
of syntax)

### React.createElement Syntax

```
const h1 = React.createElement('h1', {}, 'Hi There');
```

## \* JSX (JavaScript XML)

- using XML syntax in JavaScript
- used to design user interfaces in React
- React uses Babel behind the scene to convert the JSX syntax to `React.createElement()` function.

Markup Language : Language made with only two things i.e. tags and the content.

(Hypertext) :- Text used to link one document to another document

### Extensible Markup Language (XML)

- used as a serialization format
- making raw data into information

Example:-

```
<person>
  <age> 10 </age>
  <name> persons </name>
  <address> pune </address>
</person>
```

- HTML uses XML Syntax to design web pages.
- the tags are standardized by W3C.

### \* String Interpolation

```
const username = 'steve'
```

```
const h1 = <h1> Hello {username} </h1> // Hello steve
```

### \* Conventions to write JSX

- we <sup>XML</sup> (HTML) Syntax in JS
- the properties are using camelcase : className
- to read value of a variable use string interpolation
- array
  - to read values of an array use map function.
  - in array every element must have a unique key to identify each element uniquely.

### \* What is map & filter ?

map is used for transformations of values of an array.

filter is used to filter elements, which satisfies a particular condition.

- map is used when there is no selection required

\* When to use map

- use it only when there is no cond" required to select

- use both (map & filter) when

- we require both projection and selection is required

- Always use filter (first) and map (afterwards)

filter → map

→ using map in JSX

```
const countries = ["india", "uk", "japan", "russia", "bhutan"]
```

```
const div = countries.map(country => {  
  return <div> {country} </div>  
});
```

(But here each element does not have a unique key)

```
const div = countries.map((country, index => {  
  return <div key={index}> {country} </div>  
});
```

from older state

\* These keys are used for tracking the changes ~~represented~~ in  
to new state of virtual dom, by the diffing algorithm.

\* Props are parameters / properties passed to a functional component.

## \* Components

\* What is a React Component ?

\* Explain React Architecture.

- react has a component based architecture, which makes it more flexible and reusable.
- A component is a reusable entity which contains the user interface created using JSX and it contains state (to store data for internal use) and event handlers (used to handle events generated by the component UI).

\* A component has

- i) User Interface
- ii) State (optional)
- iii) Event handlers (optional)
- iv) Props (optional)

\* Types of components :-

i) Class Component

- a class derived from `React.Component` class
- this class implements method named `render()`.
- used a lot before react hooks to create stateful components

ii) Functional Component

- JavaScript function which returns a JSX interface
- Optionally maintains its own state (using react hooks)
- earlier it was used to create only stateless component but after introduction of react hooks today, it is also used to create a stateful component as well.
- Stateless means a component which doesn't maintain its state.

### \* Functional components vs class components

- Functional components were used to create stateless components and they are lightweight and faster
- Class components were used to create stateful components and they are heavier and slower

### \* What is props?

- Props represents properties of a component
- a component may receive a parameter named props to collect all the properties passed to component.
- a props is an object which has all the properties as key-value pairs.
- it is a way for ~~parent~~ parent to pass the information with child component.
- (it's a readonly object)
  - child is not supposed to change the props.
  - even if child changes props, the changes will not reflect in Parent's object.

### \* What is Object Destructuring?

- Object destructuring means extracting specific properties of an object and assign them as variables.

```
function person ({ props, propz }) {  
  -----  
  -----  
}
```

→ This type of destructuring will also provide intellisense when passing props

- \* When writing a component, make it loosely coupled with the structure of property object, i.e. we need to make it dependent on property name, instead of the entire object.

## \* Vite

- a build tool used to build multiple types of react e.g. react, angular, preact, svelte, etc.
- build
  - Compile the project :- minifying (reducing the file size) the JS files
  - add the external referenced packages.
  - create a bundle (collection of required files) for deployment.
- Diff b/w dependencies and devdependencies.
- Dependencies are required to run the application and they are included inside the bundle, when the application is build.  
(dist)
- Devdependencies are required to develop the application and they are not included in the bundle of built application.

## \* src might also contain other folders

- 1) components
- 2) Pages
- 3) Features / Slices (contains slices required for redux toolkit)
- 4) Services (contains services (axios functions) to connect to backend)

## \* eslint.config.js

- contains eslint configuration
- linter is a tool which is used to find syntactical error/warning

#### - index.html

- only html file in project
- gets loaded in the browser when the application starts
- never gets reloaded implicitly

#### - package.json

- contains the metadata of application
- contains •
  - name
  - version
  - scripts
  - dependencies
  - devdependencies
  - 

#### - package-lock.json

- contains ~~add~~ list of all dependencies installed from package.json.

(Server - process used to serve certain type of requests)  
(it is always a application / software)

#### \* Project dev booting

- npm run dev command gets fired
- vite starts a lite webserver on port 5173
- lite server loads a index.html file
- index.html loads the main.js

## - default export

- used to export only one entity (function/const/variable) from a file

- uses 'export default' keyword to export an entity  
↗ (import component-name from ".....")

## - export (normal / simple)

- used to export multiple entities from a file
- use keyword 'export' for all the exported entities.

ex:- export function Person() {

-----

-----

}

export function car() {

-----

}

(import) :-

import { Person } from './Person';

## \* Handling events in react

- define a event handler inside a component
- event handler
  - function which handles a required event

## \* State

- a state is an object maintained by component.
- is read-writable object.
- is a collection of key-value pairs
- is a trigger to re-render the component's UI
  - when state of a component is updated, the component re-renders its UI with updated values.

\* State members/variables gets stored in browser's memory 1/1/1

- to maintain a state object inside a component
  - class component
    - provides state object implicitly (`this.state`)
  - functional component
    - use react hooks named `useState`

## \* React Hooks

- react hook is a special function in react which starts with 'use' prefix.

|                            |                              |                            |
|----------------------------|------------------------------|----------------------------|
| - <code>useState</code>    | provided by react-router     | provided by redux          |
| - <code>useEffect</code>   | - <code>useNavigate()</code> | - <code>useSelector</code> |
| - <code>useCallback</code> | - <code>useLocation()</code> | - <code>useDispatch</code> |
| - <code>useMemo</code>     |                              | - <code>useReducer</code>  |

## \* `useState`

- used to add a member to component's state object
- accepts a initial value of the member.
- returns an array with 2 members.
  - 0<sup>th</sup> position (getter) : reference to the ~~state~~ state member to read the current value
  - 1<sup>st</sup> position (setter) : reference to the function to update the state

## \* REST

- Stands for Representational State Transfer
- \* \* - a design pattern developed on top of HTTP architecture
- it contains
  - http methods
  - http request
  - http response

- used to send data (generally in JSON) to the client.
- can be used to send data to client in different format as well.
- GraphQL
  - stands for graph query language
  - developed on top of REST
  - uses a POST method to send the query or mutation to perform the communication.
- gRPC (google Remote Procedure Call)
- BrowserRouter
  - used to add routing feature for website (browser)
  - must be parent of `<App/>` component in order to add the routing feature.
- Routes
  - collection of route objects
  - contains all possible routes of an application
- \* In Express, a Route is a mapping of a http method, the path and the handler function
- Route
  - a mapping b/w path (to launch the component) and the element (component to be launched)
- Link
  - used to navigate from one page to another page
  - present in react-router-dom package.

\* Hooks need to be called directly inside the component body (not inside any inner f<sup>n</sup> in functional component). \_/\_/\_

- ways to navigate to another component / page.

i) Static navigation

- static navigation always loads the same component every time
- can be done using `<Link>` `</Link>` tag
- no need to write any JS code.

ii) Dynamic Navigation

- navigation with JS code
- can be achieved by a react hook named 'useNavigate ()'.
- also called as conditional navigation.

## \* Component Life Cycle

- hooks are used to handle the component's life cycle events.

- life cycle events

- component mounted
- component unmounted
- component state changed
- component state member changed.

- Component mounted

- this event ~~runs~~ occurs only once when the component gets mounted
- `useEffect` hook is used for ~~this event~~ <sup>actions</sup> to take place when component mounts.

- here the dependency array comes in handy for controlling behaviour.

## - Component unmounting

- this event occurs when a component gets unmounted.
- in `useEffect` hook, we return a cleanup function.

for ex:-

```
useEffect ( () => {  
    // actions code here  
  
    // return cleanup, which runs when component unmounts  
    return () => {  
        console.log("comp. unmounted");  
    }  
});
```

## - Component State change

- required to perform an action when a component state is changed
- dependency array must be empty and not ~~provided~~

```
useEffect ( () => {  
  
    // actions code  
  
});
```

## - Component State member changed

- used to handle state change of one member.
- required to add state in the dependency array.